

Panorama: origen desde Facebook / Instagram → TIA

Notas de exploración (OpenSpec explore). Documento para análisis; no es una especificación formal ni un plan de implementación cerrado.

Hoy TIA no tiene UTM, referral ni “interés de origen”. La sesión nace con prompt + cronograma.md fijos; el API solo recibe { session_id, message }. El vínculo social → chat hay que diseñarlo encima de eso.

1. El flujo real (no es “abrir el bot dentro de IG”)

Instagram/Facebook casi nunca embeben tu chat. Lo habitual es un click a una URL de aterrizaje tuya, con parámetros en la query string.

Publicación IG/FB

| link en bio / historia / anuncio / botón del post

v

URL de aterrizaje (web o landing)

?origen=ig&interes=servicios <- acá vive el parámetro

|

v

UI de chat --POST /chat--> TiaChatbot (+ metadata de origen)

|

v

system (o primer mensaje) con contexto de ámbito

Lo que Meta te da es un click a una URL. El parámetro es de tu landing/API, no un feature nativo del post.

2. Qué modelar como “dato de origen”

Separar tres capas evita confusiones:

- Canal: instagram, facebook, directo → analytics + PING (“vino de IG”).
- Campaña / post: post_marzo_masajes, utm_campaign=... → medir qué publicación convierte.
- Ámbito de interés: servicios, actividades, eventos, maternar → orientar a la IA (no una actividad concreta del cronograma).

El foco del producto es la tercera capa: no “Yoga Postparto #8”, sino “esta persona probablemente viene por servicios / bienestar corporal”. Eso encaja con las secciones del cronograma (Actividades grupales, Agenda de Servicios, futuro Eventos).

Un vocabulario chico y estable (interes=servicios|actividades|eventos|maternar|general) es más fácil de mantener que IDs de post sueltos.

3. Cómo inyectar ese dato a la IA

Opciones principales:

- A) Bloque en el system al crear la sesión (recomendable para un MVP). El modelo lo ve en todos los turnos; no ensucia el historial visible; el PING puede reutilizar el mismo metadata.
- B) Mensaje system oculto / interno (similar a A).
- C) Primer mensaje “user” sintético (más ruidoso en la UI).

Ejemplo de bloque de origen (pista, no hecho)

Contexto de origen: la persona llegó desde Instagram, campaña X.

Ámbito sugerido: Agenda de Servicios (no una actividad concreta).

Orientá la charla hacia ese ámbito; no asumas que ya eligió “masajes”.
Si pregunta otra cosa, respondé igual.

Importante: es una pista, no un hecho. La usuaria puede llegar por un post de masajes y preguntar por yoga.
El prompt debe decir “priorizá / ofrecé primero”, no “solo hablá de esto”.

4. Dónde engancha en este repo (hoy)

```
Landing (?interes=...)
-> front guarda session_id + origin
-> POST /chat { session_id, message, origin?: {...} } <- hoy no existe
-> TiaChatbot.ask(...)
-> primera vez: system = prompt + knowledge + ORIGEN
```

Hoy existen CLI, Gradio y FastAPI (app/api.py). Gradio no lee query params de una URL pública de Meta sin un front/landing delante. En producción casi seguro hace falta una página que lea ?interes= y abra el chat. Archivos clave: src/chatbot.py, app/api.py, gradio_app.py, main.py.

5. Meta: qué tener en cuenta al armar el vínculo

- Link en bio / Linktree / ManyChat → URL con query string (lo más simple).
- Stories / ads → suelen aceptar link de destino; ahí van interes + utm_*.
- Posts orgánicos → link clickeable limitado; a menudo “link en bio” + copy “escribí SERVICIOS”.
- WhatsApp Business / DM → otro producto (no es tu FastAPI); el origen se modela distinto (referral de Meta).
- In-app browser → a veces pierde query params o cookies; capturar params en la URL al primer load y fijarlos en la sesión.

6. Riesgos y diseño

- Spoofing: cualquiera puede abrir ?interes=servicios. Sirve para orientación, no es seguridad.
- Taxonomía: el mapa interes → sección del cronograma debe vivir en un solo lugar (config o knowledge), no hardcodeado en 10 posts sin documentar.
- Primera respuesta: ¿saludo genérico o “Vi que venís por servicios...”? Lo segundo convierte mejor; cuidar el tono para no sonar stalking.
- Identificación: el origen no reemplaza nombre/teléfono; puede anticipar el ámbito y pedir ID cuando el interés se concrete.
- PING: muy útil para el equipo — Canal: Instagram · Ámbito: Servicios · Campaña: ... además de los intereses del diálogo.
- Sesión: si reusan session_id o refrescan sin params, se pierde el origen → capturar y fijar en la sesión al primer hit.
- Privacidad: no hace falta PII en la URL; interes + utm_source alcanzan.

7. Panorama de trabajo (si pasa a un change OpenSpec)

- Contrato de origen — schema mínimo: source, interest_scope, opcional campaign.
- Landing — lee query y abre chat con ese contexto.
- API — aceptar metadata en /chat (o POST /session al crear).
- Runtime — inyectar bloque ORIGEN solo al crear historial.
- Prompt — priorizar ámbito, no inventar actividad concreta, no ignorar si cambian de tema.
- PING — incluir origen en el mail.
- Ops — tabla “post → URL con params” para marketing.

Nada de eso existe hoy; el enganche natural es extender el bootstrap de sesión en TiaChatbot + payload del

API. Nombre tentativo de change: session-origin-context.

8. Preguntas abiertas

- ¿El chat en producción será web propia, Gradio, o WhatsApp?
- ¿El ámbito es solo las 3 agendas, o también temas blandos (embarazo, postparto)?
- ¿Querés que el primer mensaje lo escriba sola TIA (“Hola, vi que te interesan los servicios...”) sin que la usuaria hable primero?

Resumen en una frase

El vínculo Meta → TIA es una URL de aterrizaje con parámetros; lo que orienta a la IA es un ámbito de interés (sección del cronograma), inyectado al crear la sesión como contexto de sistema, y opcionalmente reflejado en el PING — siempre como pista, nunca como camisa de fuerza.